SMASH - MAP JSON IMPORT SPECIFICATION AND REVIEW
Unity 6 / URP / Android - project "Demolition Contracts" (FierceTigerGameJam)
Branch: Falcon/UpdateMapData Document date: 2026-08-28 Revision 2

```
WHO THIS IS FOR
    The AI/tool that generates map JSON files for this game. It documents
    exactly what the Unity importer accepts, the maths it applies, the rules
    that make a block get skipped, and a review of the four files currently in
    Assets/GameJam/Maps/.

CODE THIS DESCRIBES (source of truth, do not guess around it)
    Assets/GameJam/Scripts/Gameplay/Wall/KnockdownMapDefinition.cs   (parse + validate)
    Assets/GameJam/Scripts/Gameplay/Wall/KnockdownLayoutMapAuthoring.cs (BuildMap)
    Assets/GameJam/Scripts/Gameplay/Wall/MapConfig.cs                (registration)
    Assets/GameJam/Scripts/Config/MapProgressionConfig.cs            (rules/rewards)
    Assets/GameJam/Config/BlockDatabase.asset                        (type -> prefab)
```

0. THE FIVE THINGS TO FIX FIRST

1. cellSize AND layerDepth MUST BE EXACTLY 0.25.
   They are grid SPACING, not a scale factor. Block prefabs are a fixed
   0.25 x 0.25 x 0.25 world units and the builder explicitly forces
   localScale = Vector3.one on every instance. Any other value does not
   resize anything - it only moves blocks closer together or further apart.
   cellSize 0.0615 (map_002_footprint_test.json) makes every 0.25 m block
   overlap its neighbours by ~75%, which the physics engine resolves by
   violently pushing them apart. Because blocks start kinematic
   (section 3.1) the map looks intact when it loads and detonates the moment
   the first shot lands, which is worse than failing outright.

2. THE TOP-LEVEL "id" MUST BE UNIQUE, STABLE AND HUMAN-READABLE.
   Use "map_004", "map_007_office", not
   "models3d_rcreshryueasj1vmutscx5bmqgc3_ooo64gqd4p5fj8x56zsd".
   It is a gameplay key: MapConfig matches its entry id against it, and
   MapProgressionConfig keys rewards, clear percentage and ammo limit off the
   same id. Two files currently share one generated id, which makes them
   indistinguishable to the game.

3. USE "wall": { "wall_id": "..." } OR ACCEPT ONE RIGIDBODY PER BLOCK.
   The builder defaults to WallGroupingMode.NamedOnly: blocks without a
   wall_id are never merged. 3285 loose rigidbodies (map_002) will not run on
   a mid-range Android phone. Group flat runs - facades, floors, slabs - into
   named walls; leave only the blocks that should fly apart individually.

4. DO NOT AUTHOR FLOATING BLOCKS.
   Each layer is an independent vertical slice; layers do NOT support each
   other. A block at y=N is only held up by a block at y=N-1 IN THE SAME
   layer. 24-28% of the blocks in the two generated files have nothing under
   them.
   Note that they do NOT fall at load: blocks start kinematic (section 3.1).
   They hang frozen in mid-air forever, because the support cascade only
   releases blocks ABOVE a destroyed block and a floating block has no
   supporter to release it. They also keep counting as "remaining", so they
   hold the map below 100% and can put the clear reward out of reach.

5. KEEP THE MAP INSIDE THE BUDGET.
   Blocks and layers scale with the level's role - see the table in section 7
   (50-120 blocks for a tutorial, up to 400 for the boss map). Grid width
   <= 16, height <= 14. Detail beyond that costs frame time, not fun.

1. HOW UNITY READS THE FILE

```
The file is parsed with UnityEngine.JsonUtility, which is much stricter and
dumber than a normal JSON parser. Consequences that matter:

  * The top level must be a JSON OBJECT. A top-level array is rejected.
  * Field names must match the C# fields EXACTLY and are case-sensitive.
  * Unknown keys are silently ignored - they will not error, they will just
    do nothing. Do not invent keys and assume the game reads them.
  * There is no null / missing distinction. A missing object becomes an empty
    instance, not null. This is why an absent "wall" reads as
    wall.wall_id == "" rather than wall == null.
  * Numbers must have the right kind. int fields (width, height, level,
    position.x, position.y) must be written as integers, never "3" as a string
    and never 3.0. float fields (cellSize, layerDepth, rotation) accept both.
  * No dictionaries/maps. Everything is an object with fixed fields or an array.
  * Comments, trailing commas and NaN are not allowed.

After parsing, KnockdownMapDefinition.TryParse rejects the whole file (nothing
is built, one error is logged) when:

    "The map JSON is empty."
    "The map JSON could not be parsed: ..."
    "The map JSON is missing a grid or layers block."
    "The map declares schemaVersion N, but only 1 is supported."
    "The map grid needs a positive width and height."
    "The map grid needs a positive cellSize and layerDepth."
```

2. SCHEMA (schemaVersion 1)

```
ROOT OBJECT
  schemaVersion  int      required  Must be exactly 1.
  id             string   required  Unique map key. Must equal the id used in
                                    MapConfig for this map.
  grid           object   required  See GRID.
  layers         array    required  See LAYER. At least one.

GRID
  width          int      required  Number of columns (X). Must be > 0.
  height         int      required  Number of rows (Y, upward). Must be > 0.
  cellSize       float    required  MUST BE 0.25. Metres per cell in X and Y.
  layerDepth     float    required  MUST BE 0.25. Metres between layers in Z.

LAYER
  level          int      required  Z index of this depth slice. 0 = front,
                                    increasing = further back. Use 0..N-1 with
                                    no gaps and no duplicates.
  blocks         array    required  See BLOCK.

BLOCK
  id             string   required  Unique within the file. Convention used by
                                    the project: "f{level}_c{x}_r{y}".
  type           string   required  Must exist in BlockDatabase (section 5).
  position       object   required  { "x": int, "y": int }
  rotation       float    required  0, 90, 180 or 270 only.
  wall           object   optional  { "wall_id": "some_name" } - see section 6.

POSITION
  x              int      required  Column, 0 .. grid.width - 1.
  y              int      required  Row, 0 = standing on the floor,
                                    up to grid.height - 1.

MINIMAL VALID FILE
```

```
{
  "schemaVersion": 1,
  "id": "map_004",
  "grid": { "width": 4, "height": 3, "cellSize": 0.25, "layerDepth": 0.25 },
  "layers": [
    {
      "level": 0,
      "blocks": [
        { "id": "f0_c0_r0", "type": "concrete_1x1", "position": { "x": 0, "y": 0 }, "rotation": 0,
          "wall": { "wall_id": "base" } },
        { "id": "f0_c1_r0", "type": "concrete_1x1", "position": { "x": 1, "y": 0 }, "rotation": 0,
          "wall": { "wall_id": "base" } },
        { "id": "f0_c0_r1", "type": "brick_1x1",    "position": { "x": 0, "y": 1 }, "rotation": 0 },
        { "id": "f0_c1_r1", "type": "glass_1x1",    "position": { "x": 1, "y": 1 }, "rotation": 0 }
      ]
    }
  ]
}
```

3. COORDINATE SYSTEM AND PLACEMENT MATHS

```
AXES
    X   grid column, position.x, runs left to right, spaced by cellSize.
    Y   grid row, position.y, runs UPWARD from the floor, spaced by cellSize.
        y = 0 sits with its base on the ground plane.
    Z   depth, comes from layer.level, spaced by layerDepth. Level 0 is the
        slice nearest the player/cannon, higher levels are further back.

So a "layer" is NOT a floor of a building. It is a vertical slice at one depth.
A tall building is many rows (y) inside few layers; a deep building is many
layers.

EXACT PLACEMENT (from KnockdownLayoutMapAuthoring)

    origin.x = -((grid.width - 1) * cellSize) * 0.5      // when centerGrid is on
    origin.z = -(maxLevel * layerDepth) * 0.5

    world.x = origin.x + (position.x + (footprint.x - 1) * 0.5) * cellSize
    world.y =            (position.y + footprint.y * 0.5)       * cellSize
    world.z = origin.z + (level      + (footprint.z - 1) * 0.5) * layerDepth

Block pivots are centred, which is why the half-footprint offsets appear. The
structure is centred on X and Z and stands on Y = 0. Nothing here scales the
prefab; cellSize only stretches or compresses the lattice.

ROTATION
    rotation is applied as Quaternion.Euler(0, 0, rotation) - a spin about Z,
    inside the block's own slice. Its real job is to choose the FOOTPRINT
    orientation: 0 and 180 keep the authored width x height, 90 and 270 swap
    them. For a 1x1 block every rotation looks and behaves identically, so
    emitting random 90/270 values on 1x1 blocks is harmless noise - prefer 0.
    Only brick_2x1 is affected today (2x1 at rotation 0, 1x2 at rotation 90).

3.1 PHYSICS ACTIVATION MODEL - READ THIS BEFORE REASONING ABOUT SUPPORT

    Blocks do NOT simulate when the map loads. KnockdownBlock.Awake sets
    isKinematic = true and useGravity = false on every block. A block starts
    moving only when it is Activated, which happens in exactly three ways:

      a) it is hit by the projectile,
      b) it is struck by an already-activated block at more than
         collisionActivationVelocity (1.75 m/s by default),
      c) a block BELOW it is activated and the support cascade releases it
```

```
          (SupportCascadeMode.ColumnAbove walks the cells directly above the
          destroyed block's footprint, in the same layer).

    Two consequences the map author has to design around:

    * A FLOATING BLOCK NEVER FALLS ON ITS OWN. It has no block beneath it, so
      path (c) can never reach it. It hangs frozen in the air until the player
      happens to shoot it. It is not a physics failure, it is a permanent
      visual artefact plus a completion blocker.

    * OVERLAPPING BLOCKS LOOK FINE UNTIL THEY ARE TOUCHED. Interpenetrating
      kinematic bodies sit still. The first activation turns the overlap into a
      very large separation impulse.

    Blocks inside a NAMED WALL are a different case: the wall is one rigidbody,
    so its blocks are held by the wall as a whole. A wall only needs its own
    footing, not a supporting cell under every block it contains.

SUPPORT / GRAVITY
    Support is resolved per physics body, not per cell:
      - a loose block at (x, y) needs a block or a wall occupying (x, y-1) in
        the SAME layer;
      - a named wall needs a footing under the wall as a whole.
    Adjacent layers are separate slices and never hold each other up.
```

4. HARD RULES - WHAT MAKES A BLOCK OR A FILE FAIL

```
WHOLE FILE REJECTED (nothing is built)
    schemaVersion != 1
    grid or layers missing
    grid.width <= 0 or grid.height <= 0
    grid.cellSize <= 0 or grid.layerDepth <= 0
    malformed JSON

BLOCK SKIPPED, REST OF MAP STILL BUILDS
    "A block on level L has no position and was skipped."
        -> position object missing.

    "Block {id} uses type "X", which the block database does not map to a prefab."
        -> type is not one of the four in section 5. Logged as an ERROR.

    "Block {id} has rotation R, which is not a multiple of 90."
        -> only 0/90/180/270 are accepted. Logged as an ERROR.

    "Block {id} covers cell (cx, cy) on level L, which is outside the WxH grid.
     Skipped."
        -> the block, INCLUDING its full footprint, must fit. A brick_2x1 at
           x = width-1 with rotation 0 covers width, which is out of bounds.

    "Block {id} wants cell (cx, cy) on level L, which is already taken. Skipped."
        -> two blocks claim the same cell. Cells are reserved per
           (x, y, level); a block whose footprint has depth > 1 also reserves
           the layers behind it.

VALIDATION AT REGISTRATION TIME (MapConfig.OnValidate)
    "{asset}: map "X" has no JSON assigned."
    "{asset}: map "X" has invalid JSON. ..."
    "{asset}: entry id "X" does not match the id "Y" inside {file}."
        -> the top-level "id" in the JSON and the id typed in MapConfig must be
           the same string.
```

5. BLOCK CATALOGUE (BlockDatabase.asset)

type	footprint	world size (m)	mass	hit points	material
concrete_1x1	1 x 1 x 1	0.25 x 0.25 x 0.25	2.0	6	concrete
brick_1x1	1 x 1 x 1	0.25 x 0.25 x 0.25	1.2	3	brick
brick_2x1	2 x 1 x 1	0.50 x 0.25 x 0.25	2.4	5	brick
glass_1x1	1 x 1 x 1	0.25 x 0.25 x 0.25	0.6	1	glass

* These four are the ONLY valid values of "type". Anything else is an error.
* steel was considered and dropped - do not emit it.
* brick_2x1 is currently unused by the generated files. It is the cheapest way to halve the object count on long brick runs, so prefer it over two brick_1x1 wherever a run is even in length.
* Design intent per material: glass is the breakable filler (windows, curtain walls), brick is the ordinary structure, concrete is the frame that should survive longest.

6. WALL MERGING - "wall": { "wall_id": "..." }

```
WHY IT EXISTS
    Every unmerged block is one GameObject + one Rigidbody + one BoxCollider.
    A merged wall is a single body with a single collider and a single mesh
    that shatters back into its blocks when it is knocked down. Merging is the
    main lever on performance and it is the MAP's decision, not the engine's.

    IMPORTANT LIMIT: merging lowers the STANDING cost, not the PEAK cost. A
    wall keeps a manifest of the blocks it replaced and spawns them back when
    it comes apart. A 400-block map merged into 30 walls still ends up with
    roughly 400 bodies if the player destroys everything. This is why the total
    block count is capped as well as the loose count - the loose budget governs
    the opening frame, the total budget governs the worst frame.

MODE
    The authoring component defaults to WallGroupingMode.NamedOnly:
      None             never merge.
      NamedOnly        merge only blocks that carry a wall_id.   <- DEFAULT
      NamedAndDetected NamedOnly plus a legacy fallback that auto-merges
                       same-type runs (>= minimumWallCells, default 3) inside
                       one layer, for maps written before wall ids existed.

RULES FOR A NAMED WALL
    * Blocks in one wall may span different types and different layers. The
      wall answers to the FIRST block's material, and its hit points are the
      SUM of its blocks' hit points.
    * A wall_id used by only one block is built as a plain block, with the
      warning: "Wall "X" has only 1 block(s), so it is built as a plain block."
    * Blocks in one wall_id MUST TOUCH. If they form two or more disconnected
      clusters, the builder splits them into one wall per cluster and warns:
      "Wall "X" is N groups of blocks that do not touch each other...".
      This matters because a single box collider stretched over a gap would
      stop shots in mid-air.
    * wall_id strings are compared with ordinal equality - "Front" and "front"
      are different walls.

HOW TO USE IT WHEN GENERATING
    Merge: floor slabs, roof slabs, uninterrupted facade panels, solid
           back walls, foundation rows - anything the player is meant to knock
           over as one piece.
    Leave loose: window frames, ledges, decorative parapets, anything that
           should scatter into individual chunks, and anything the player is
           meant to punch a hole through.
    Rule of thumb: a good map has more cells inside walls than outside them.
```

7. PERFORMANCE BUDGET (Android is the target)

```
Blocks per map (total)         see table below   hard ceiling ~500
Loose (unmerged) rigidbodies   <= 150            these simulate from frame 1
Named walls per map            5 - 40
Layers per map                 2 - 12
grid.width                     4 - 16
grid.height                    3 - 14

The total is not one flat number - it scales with the level's role. Tutorial
maps are meant to be small; a 60-block floor would only pad them.

  level band                blocks     layers    loose bodies
  ----------------------    --------   -------   ------------
  tutorial / level 1-3       50 - 120   2 -  5     15 -  35
  mid / level 4-6           120 - 220   4 -  7     25 -  60
  hard / level 7-9          220 - 350   5 - 10     45 - 100
  boss / level 10           280 - 400   7 - 12     70 - 125

Purpose-built test maps (map_001, map_003_wall_groups) sit below all of this
on purpose and are not held to it.

A map is not better because it has more blocks. Silhouette, material
contrast and a clear weak point read at a glance; 3000 quarter-metre cubes
read as noise and drop the frame rate.
```

8. REVIEW OF THE FILES CURRENTLY IN Assets/GameJam/Maps/

```
file                          id                grid       cell    layers blocks
--------------------------- ----------------- ---------- ------- ------ ------
map_001.json                map_001           4 x 3      0.25    2      18
map_003_wall_groups.json    map_003_wall_..   6 x 4      0.25    2      20
test_03.json                models3d_rcre..   10 x 9     0.25    14     471
map_002_footprint_test.json models3d_rcre..   26 x 23    0.0615  32     3285
```

map_001.json STATUS: GOOD

```
Hand-authored reference. 18 blocks, no floating blocks, correct cell size,
readable id, mixes all three materials, uses rotation 90 on two blocks.
Treat this file as the shape of a well-formed map.
```

map_003_wall_groups.json STATUS: GOOD

```
The reference for wall grouping: three named walls (front_concrete,
front_brick, deep_pillar) of 4 blocks each out of 20. This is the only file
that demonstrates "wall": { "wall_id": ... }. Copy this pattern.
One block sits with nothing beneath it - minor, likely intentional.
```

test_03.json STATUS: BUILDS, PLAYS BADLY

```
PASSES the parser: 0 hard errors, schema and cell size correct, all types
valid, no cell collisions, everything inside the grid.

Problems:
P1  id is "models3d_rcreshryueasj1vmutscx5bmqgc3_ooo64gqd4p5fj8x56zsd" and is
    SHARED with map_002_footprint_test.json. Two files cannot claim one map
    key. Rename to map_004 (or similar) and make it unique.
P2  132 of 471 blocks (28%) have nothing supporting them in their own layer -
    entire glass rows at y = 6, 7, 8 and isolated concrete at y = 1. Per
    section 3.1 these do not fall; they hang in the air permanently, cannot
    be reached by the support cascade, and keep the map from ever reading
    100% cleared unless the player shoots each one.
```

```
P3  No wall_id anywhere: 471 independent rigidbodies. Above the loose-body
    budget. The solid concrete mass and the glass curtain rows are obvious
    merge candidates and would cut this to well under 100 bodies.
P4  264 of 471 blocks carry rotation 90/180/270 on 1x1 cubes, where rotation
    has no effect. Harmless, but it is noise in the file and hides the cases
    where rotation would matter. Emit 0 unless the block is brick_2x1.
P5  14 layers x 471 blocks is at the top of the budget for a 2.5 x 2.25 x
    3.5 m structure. Consider fewer, denser layers.
```

map_002_footprint_test.json STATUS: BROKEN

```
C1  CRITICAL: cellSize and layerDepth are 0.0615 instead of 0.25. Prefabs are
    not scaled by cellSize, so 3285 blocks of 0.25 m each are placed on a
    0.0615 m lattice. Every block interpenetrates its neighbours on all three
    axes. It will look correct on load and blow apart on the first shot.
    It looks as though the generator tried to fit a 26 x 23 x 32 model into
    the physical size of a small map by shrinking the cell. The correct fix is
    to REDUCE THE CELL COUNT, not the cell size.
C2  Same duplicated generated id as test_03.json.
C3  3285 blocks / 32 layers - roughly 8x the intended ceiling. At 0.25 m per
    cell this would be a 6.5 x 5.75 x 8.0 m structure, far larger than the
    playfield and the cannon's range.
C4  782 blocks (24%) are unsupported inside their layer.
C5  No wall_id anywhere.
Recommendation: do not import as-is. Regenerate at cellSize 0.25 with the
grid reduced to roughly 12 x 10 x 6 and named walls on every flat run.
```

9. CHECKLIST FOR THE GENERATOR

```
[ ] schemaVersion is exactly 1
[ ] id is short, unique, human-readable ("map_00N"), and matches the id that
    will be typed into MapConfig
[ ] cellSize == 0.25 and layerDepth == 0.25
[ ] grid.width and grid.height actually cover the highest x and y used
[ ] layer levels are 0..N-1, each appearing exactly once
[ ] every block id is unique within the file, formatted f{level}_c{x}_r{y}
[ ] every type is one of concrete_1x1 / brick_1x1 / brick_2x1 / glass_1x1
[ ] rotation is 0 for every 1x1 block; 0/90/180/270 for brick_2x1
[ ] no two blocks claim the same (x, y, level), footprints included
[ ] support is checked AFTER grouping, per physics body: every loose block at
    y > 0 has something beneath it in the same layer, and every named wall has
    a footing. No cell-by-cell support test on blocks inside a wall.
[ ] flat runs are grouped with "wall": { "wall_id": "..." }, and every
    wall_id is used by at least two TOUCHING blocks
[ ] no wall_id mixes materials (the wall takes the FIRST block's material and
    the SUM of all its hit points, which is hard to read in play)
[ ] total blocks and layers within the level band's row in section 7,
    loose rigidbodies <= 150
[ ] the file is plain JSON: no comments, no trailing commas, ints written as
    ints
```

10. WHAT THE GAME DOES WITH THE FILE AFTER IMPORT

```
1. The .json is dropped into Assets/GameJam/Maps/ and imported as a TextAsset.
2. It is registered in Assets/GameJam/Config/MapConfig.asset as one MapInfo:
     id            - must equal the JSON's top-level "id"
     displayName   - shown in the UI
     mapJson       - the TextAsset
     clearedImage  - optional Sprite shown on the cleared screen
3. Its rules are added to Assets/GameJam/Config/MapProgressionConfig.asset as
   one Entry, keyed by the SAME id:
     mapId                 same string as above
```

```
    requiredClearPercent  0..1, share of the structure needed to pass
    passMapRewardId       RewardConfig id granted on first pass
    clearMapRewardId      RewardConfig id granted on first 100% clear
    bulletPickLimit       total ammunition the player may bring in
4. MapSelection picks the map; KnockdownLayoutMapAuthoring.BuildMap() parses
   it, reserves cells, groups named walls and instantiates the structure;
   LevelRunController reads the progression entry for the pass threshold, the
   ammo budget and the rewards.

The generator only has to produce step 1 correctly. Steps 2 and 3 are done in
Unity, but they depend on the "id" being right, which is why it matters more
than it looks.
```

END OF DOCUMENT